

Environmental Impact Estimate

Sentinel Nexus — AI Training & Inference Footprint

RW

June 2026

Contents

Purpose	2
Method	2
Estimated footprint (planning)	2
Tracking & refinement (MS-2.12-003)	2
Posture	3

Purpose

Per NIST AI 600-1 **MS-2.12** (environmental impact and sustainability) and **2.5** (Environmental Impacts), this document records the platform’s approach to estimating and bounding the energy/carbon footprint of AI training and inference. It addresses control item **NC-6**. NIST notes there is no single agreed method to estimate GAI environmental impact; the figures here are **order-of-magnitude planning estimates**, to be refined with measured data.

Method

Footprint varies by activity (pre-training vs fine-tune vs inference), modality, hardware, and task. Nexus does **not** pre-train foundation models — it **fine-tunes** (QLoRA / DPO adapters) and runs inference, which is materially lower-impact than training a transformer from scratch (NIST cites ~300 SF-NY round-trip flights of CO₂ for a single from-scratch transformer pre-training — an envelope Nexus never enters).

Estimated footprint (planning)

Activity	Driver	Estimate basis	Mitigation
Fine-tune (Models B/C/D)	A100-class GPU-hours per training run	Bounded LoRA/QLoRA + ZeRO-3 sharding; runs are infrequent (RSI-gated, not continuous)	Adapter-only training; early-stop + tier-0 canary abort doomed runs before full cost
Model A training	CPU minutes (LSTM-AE ~1 MB)	Negligible	CPU-only, tiny model
Inference	Sustained GPU for vLLM (Models B/C/D) + CPU for Model A	Per-verdict token/compute; the Layer-1 math pre-filter avoids invoking the swarm on benign traffic	Math tripwire pre-filter; distillation/quantization (4-bit NF4); ONNX export for Model A
Corpus generation	CPU	Negligible (synthetic generators)	—

Key efficiency lever: the 3-layer architecture is itself the primary control — deterministic Layer-1/2 filtering means the expensive Layer-3 LLM swarm only runs on a small fraction of events, avoiding generative inference on the bulk of telemetry.

Tracking & refinement (MS-2.12-003)

- **Per-run energy/carbon accounting is now implemented in code (control NC-11):** `agents.controls.estimate_inference_energy` (energy = power ×

time × PUE; carbon from a grid-intensity factor) plus the agents/energy-accounting.py ledger (record_run + totals) turn the order-of-magnitude estimates below into measured per-run figures the MLOps metric plane rolls up. Proven by test_ai_controls.py::TestInferenceEnergy + test_nist_controls_wave4.py::TestEnergyAccounting.

- Capture measured GPU energy per training run from the scheduler/host telemetry and record it in the RSI ledger alongside gate_scores.
- Account inference compute via the existing per-investigation efficiency metrics (turns, llm_calls, tokens_est, wall_ms) planned in the MLOps maturation metric plane (WS-A, InvestigationMetrics) — these feed the NC-11 estimator’s power×time inputs.
- Verify trade-offs between inference-time and additional training-time resources when proposing a quantized/edge variant.
- Address green-washing: any carbon-offset claim must be evidence-backed.

Posture

For a sovereign, on-prem deployment the dominant footprint is **inference**, bounded by the pre-filter architecture and quantization. Training is intermittent and adapter-scoped. The platform’s environmental risk is therefore **low-moderate**, and the controls above keep it measurable and bounded; precise per-run measurement is the remaining refinement (folded into the MLOps maturation metric plane).